

UAVX ArmQ — ExcessLift Escape Ladder + Premerlani Wind Estimator (Implementation Report)

Implements the “two-source climbing detector” and the “WP-exit escape ladder” that the ExcessLift protection was previously missing, plus a production wind estimator (Premerlani wind-triangle, iNav model) with altitude/turbulence-aware confidence. Rationale, design, and code locations are given per change.

Scope: FC C source only (UAVXArmQ/src/). No GCS changes. **Build:** FC recompile is the user’s job (per AGENTS.md); the assistant does not compile. Emulation verification is via USE_THERMALS in emu.c.

1. Two-source climb detection — control.c CheckAltHoldAlarm()

Problem

F.ExcessLift (the control-loss detector) keyed climb on **baro** ROCTrack **alone**. In non-hydrostatic lift (thermal / cumulus — air is not in hydrostatic equilibrium) the barometer’s model of “height from pressure” is exactly what a rising/dropping airmass fakes: a thermal can hold the aircraft up while the pressure-based ROC shows nothing, or worse, the two disagree. A baro-only detector cannot reliably distinguish “uncommanded climb against a pinned throttle floor.”

Solution

Climb is now confirmed from two **independent** sources:

```
boolean GpsClimbVote;
real32 GpsClimbRate;
if ((GPS.fix >= 3) && F.ValidGPSVel) {
    GpsClimbRate = -GPS.velD;           // NED "down": negate => climb +ve
    GpsClimbVote = GpsClimbRate > 0.2f;
} else {
    GpsClimbVote = true;                // GPS unusable - defer to baro
}
```

```
F.ExcessLift = !F.Soaring
    && (NavState != UsingThermal) && (NavState != BoostClimb)
    && (AltHoldThrComp <= -(pMaxAltHoldThrComp * 0.75f))
    && (ROCTrack > 0.2f)
    && GpsClimbVote;
```

- **Baro vote:** ROCTrack > 0.2 m/s (KF vertical velocity).
- **GPS vote:** Doppler vertical rate GPS.velD (NED) converted to climb via -GPS.velD > 0.2 m/s, gated by GPS.fix>=3 && F.ValidGPSVel.
- ****Both must agree above threshold** — a transient baro plate or a GPS blip alone cannot trip the flag.
- **GPS-unusable platforms:** GpsClimbVote defaults to true, so baro-only detection still works (a loss-of-control detector must not be held hostage to a GPS velocity gate).

Same exclusion rules as before: soaring, thermalling, and boost-climb never set it, because they *intend to climb*. ExcessLift remains a *loss of alt-control authority*, not a pilot altitude limit and not a thermal detector (the soar module owns that role).

2. Wind estimation — UAVXArmQ/src/wind/wind.c

Rationale (why this model)

The earlier research ([wiki/docs/Soaring_Research_ArduPilot_vs_iNav.md](#)) concluded that iNav's **Premierlani wind-triangle** was the best well-regarded open-source estimator for turbulence rejection. It was chosen because:

- **Turbulence rejection by construction.** It only integrates samples while the *fuselage* (body x-axis) direction is actually changing (a coordinated turn or pitch manoeuvre). Samples are taken **relative to the aircraft's own attitude motion**, so turbulent gusts — which move the *airmass*, not the *fuselage* — contribute far less than they would to a naive ground-track subtraction.
- It needs only GPS ground velocity + attitude — no airspeed sensor, no pitot-lag error.
- It is the same model iNav ships in production.

Architecture & confidence model

Inputs (throttled to 10 Hz by WIND_UPDATE_INTERVAL_MS): - GPS ground velocity raw Doppler GPS.velN/E/D (the geometry needs the *change across the turn*, not a smoothed kinematic velocity). - Relative-home altitude (global Altitude). - Fuselage direction = body x-axis in NED = first column of the existing Body2World DCM from quaternion $q0..q3$: $Fus = \{1-2(q2^2+q3^2), 2(q1q2+q0q3), 2(q1q3-q0q2)\}$

Baseline re-captures (LastFus, LastGndVel) on first run or after 10×100 ms of no motion, so yaw drift doesn't bias the difference.

Turn sample ($diffLenSq > WIND_MIN_TURN_LEN_SQ = 0.04$, i.e. >0.2 rad of fuselage turn): with fDiff the fuselage change and gDiff the GPS-velocity change: - eq. 6: airspeed $v = |gDiff|/|fDiff|$ (velocity change per unit rotation). - eq. 9: rotation angle = $atan2(...)$ between the two changes. - eqs. 10-12: wind = (ground-velocity **sum** - **airspeed-rotated fuselage sum**)/2**.

A **spike filter** rejects samples jumping $>WIND_SPIKE_GATE$ (4 m/s) from the current estimate — the same gate iNav uses; it eats GPS blips.

Adaptive blend (the key improvement):

```
alpha = WIND_ALPHA_SETTLED + (WIND_ALPHA_LEARN - WIND_ALPHA_SETTLED)*(1 - conf)
Wind += alpha * (sample - Wind)
```

- Unconfident $\rightarrow$ alpha = 0.10 (fast-learn).
- Settled $\rightarrow$ alpha = 0.02 (reject noise).

Steady turbulence is averaged out once trusted; a genuinely new regime (e.g. after an altitude change) re-learns quickly because confidence has decayed.

Confidence model (NEDConfidence()):

```
conf = ValidityScore - 0.05*timeSinceUsableTurn_s - 0.02*|Alt - EstAltitude|
```

- **Time decay** (0.05/s): the estimate goes stale with age of the last turn.
- **Altitude decay** (0.02/m): wind at a different altitude can be completely different (wind shear / veering), so a stale-at-altitude estimate is down-weighted and re-learning is enabled by the higher adaptive gain.
- ValidityScore bumps toward 1 on every accepted sample, capped 0-1.

Publish gate (F.WindEstValid): only when InFlight AND GPS usable AND $\text{conf} \geq 0.5$ && $\text{Speed} > 0.3$ m/s. Consumers (soaring drift, wind-comped nav, telemetry) can decide whether to trust it.

MC vs FW split: - The wind-triangle is a **fixed-wing** model (coordinated flight — velocity stays along the fuselage). For CAT_MR the body x-axis is its *bearing*, but the craft can translate in any direction (strafe / orbit / pos-hold corrections), making the triangle invalid — the same gate iNav uses. $\text{pAFTTypeCategory} \neq \text{CAT_FW} \rightarrow$ skip the triangle and go to the hover branch. - **Hover branch** (both airframes, $\text{gs} < \text{WIND_HOVER_GS} = 1.0$ m/s): a hovering craft has no forward speed, so its GPS drift is a *direct* wind observation — N/E drift is blended with $\alpha = 0.10$ and adds +0.1 to ValidityScore. MC in translation holds the last good N/E estimate and lets confidence decay.

Constants (wind.c:54-77): $\text{WIND_MIN_TURN_LEN_SQ}=0.04$, $\text{WIND_ALPHA_SETTLED}=0.02$, $\text{WIND_ALPHA_LEARN}=0.10$, $\text{WIND_SPIKE_GATE}=4.0$, $\text{WIND_VALIDITY_DECAY_S}=0.05$, $\text{WIND_VALIDITY_ALT_DECAY}=0.02$, $\text{WIND_CONFIDENCE_VALID}=0.5$, $\text{WIND_HOVER_GS}=1.0$, $\text{WIND_HOVER_ALPHA}=0.10$, $\text{WIND_UPDATE_INTERVAL_MS}=100$, $\text{WIND_MIN_SATS}=5$, $\text{WIND_MIN_SPEED}=0.3$.

Files

- src/wind/wind.c — new estimator (adapted from iNav; attribution header).
- src/wind/wind.h — moved from src/wind.h (WindStruct, EstimateWind, Wind).
- src/UAVX.h:150 — include updated to "wind/wind.h".
- uavxarm-v3-gke.c:85 — EstimateWind() call (post-net loop).
- telem.c:1153 — publish Wind.* when F.WindEstValid (pre-existing consumer now gets a real path).

3. Escape ladder for alt-closure failure — auto.c WPAltFail

Problem

WPAltFail / WPProximityFail were **dead NavStates** — declared but never entered. Real WPNav altitude closure is gated solely by CheckProximity $\rightarrow$ F.WayPointAchieved (nav.c:97-108), which needs **centring AND the altitude window** (Alt.Error within ProximityAlt). In a thermal with alt-hold pinned to the floor, the plane could circle a WP forever holding an unclosable window while ExcessLift stays set — there was **no exit**.

Ladder

AcquiringAltitude now checks F.ExcessLift **before** accepting WayPointAchieved:

```
if (F.ExcessLift) {
    InitiateLiftEscape();
    break;
}
```

InitiateLiftEscape() (auto.c) synthesises an EscapeWP 200 m on a bearing perpendicular to the current course leg ($\text{Nav.WPBearing} + 90^\circ$), commands SetDesiredAltitude(current) to hold altitude while translating, arms a distance-timeout ($\sim \text{excursion} / \text{Nav.MaxVelocity} * 1000 + 5$ s), and sets NavState = WPAltFail.

The WPAltFail case navigates EscapeWP, then re-assesses: - $\text{!F.ExcessLift} \rightarrow$ *localised* thermal — LiftEscapes = 0, NextWP() resume. - F.ExcessLift **persists at timeout/past** $\rightarrow$ *generalised* lift that lateral travel can't outrun — count LiftEscapes++; if it reaches $\text{MAX_LIFT_ESCAPES} = 3 \rightarrow$ InitiateRTH() (abandon mission, return home), else NextWP().

This is the “escape-then-return” ladder: localised lift resumes the mission after one lateral exit; persistent lift defaults to RTH after three. `LiftEscapes` is reset on a clean exit, giving hysteresis against a gust-driven abort.

FSM hygiene

- `WPPproximityFail` is left as a bare skip stub (it is a lateral-position fail, not a lift).
 - We deliberately **do not call** `RefreshNavWayPoint()` **in the fail state** — it calls `SetDesiredAltitude(WP)` and `NextWP()`. The ladder uses the isolated `EscapeWP` copy so mission WP order and the escape altitude setpoint stay intact.
 - `MAX_LIFT_ESCAPES = 3` is a conservative starting value.
-

4. Testing via the emulator

`UAVX.h:53` enables `USE_THERMALS` (two Gaussian updraft cores at (N200,E200) $r=50$ strength 4 and (N300,E50) $r=65$ strength 7). In emulation `FakeROC` drives **both** votes of the new two-source gate: `- ROC = FakeROC (emu.c:571) → the baro ROCTrack path`; `- GPS.velD = -FakeROC (emu.c:555) → the GPS NED vertical-rate path`.

So inside a thermal core both `F.ExcessLift` sources agree, and the escape ladder can be exercised end-to-end (entry → localised→resume, generalised→RTH) without touching baro/GPS physics.

5. Source attribution (required)

- `wind/wind.c` header: *Based originally on work by William Premerlani (wind- triangle estimation) and the iNav project (flight/wind_estimator.c). Adapted 2026-08-06 by GKE.*
 - `AGENTS.md` carries the mandatory Source Attribution section.
-

6. Remaining / follow-up

- **User:** validate the `auto.c` escape ladder in the simulator; re-tune `MAX_LIFT_ESCAPES` if needed.
- **Optional later:** wire the new `Wind/Confidence` into the `soar.c:157–166` drift block (currently static) now that a real value is computed.
- Keep `pMaxAltHoldThrComp` default 25% and the 0.75 floor ratio A as-is unless tuning says otherwise.

Sources / Files

- `UAVXArmQ/src/wind/wind.c`, `wind/wind.h`, `UAVX.h`, `control.c`, `auto.c`, `uavx-arm-v3-gke.c`, `telem.c`, `emu.c`.
-

7. Wind-compensated navigation — crab feedforward into the nav loop (2026-08-20)

Context

The estimator (Section 2) published `Wind.Est/Confidence` but nothing in the nav loop consumed it: `auto.c` used it only for the ground-speed/loiter decision, and `telem` for display. In steady crosswind a fixed wing steered purely on `Turn = MakePi(Heading - Nav.WPBearing)` (`nav.c`) sits in the wind and lets the cross-track error build the S-curve on the home leg; the position integrator has to fight the wind that should instead be a *known feedforward*.

What changed (A — FW/VTOL track tracking)

Navigate's fixed-wing branch now targets a **wind-crabbed heading**:

```
Nav.DesiredHeading = WindCrabHeading(Nav.WPBearing);
Turn = MakePi(Heading - Nav.DesiredHeading);

WindCrabHeading() (nav.c):

real32 WindCrabHeading(real32 TrackBearing) {
    real32 Vg, Wx, Crab = 0.0f;

    Vg = sqrtf(Sqr(Nav.C[eNorthC].Vel) + Sqr(Nav.C[eEastC].Vel));
    Wx = Wind.Est[eEastC] * cosf(TrackBearing)
        - Wind.Est[eNorthC] * sinf(TrackBearing);
    if ((Vg > NAV_WIND_MIN_GS_MPS)
        && (Wind.Confidence >= NAV_WIND_CONF_MIN)
        && (GPS.hAcc <= GPS_MIN_HACC)) {
        Crab = asinf(Limit1(Wx / Vg, 1.0f));
        Crab = Limit1(Crab, NAV_MAX_WIND_CRAB_RAD);
    }
    return (Make2Pi(TrackBearing - Crab));
}
```

- **Geometry:** `Wx` is the wind component perpendicular to the track (positive = blowing the craft right of it); the nose must point upwind of the track by `Crab = asin(Wx/Vg)`, so target heading = bearing - Crab. `Vg` is the satellite-derived GPSKF ground velocity (inertial.c:516).
- **Gates (all must hold, else Crab = 0 → behaviour identical to current):** ground speed > `NAV_WIND_MIN_GS_MPS` (2), `Wind.Confidence` ≥ 0.5 (`NAV_WIND_CONF_MIN`, matches `WIND_CONFIDENCE_VALID`), and `GPS.hAcc` ≤ `GPS_MIN_HACC` (5 m) so a degraded fix cannot steer the nose.
- **Bounded:** crab capped at `NAV_MAX_WIND_CRAB_RAD` = 30° (`asinf` also clamped via `Limit1`), so a bad estimate costs at most a bounded heading offset, never a commanded turn beyond a normal bank.
- **Scope:** only inside `Navigate`/autonomous states — never in `PIC` (`PassThru` bypasses nav entirely), and thermal/takeoff-hold special cases are untouched.

What changed (B — orbit/loiter)

`DoOrbit` now also sets `Nav.DesiredHeading = WindCrabHeading(Nav.WPBearing)` instead of the bare bearing, so a loiter/orbit crab-corrects continuously like `iNav`'s loiter wind compensation instead of drifting downwind. The orbit radius law itself (`DoOrbit` tangential velocity) is unchanged.

Measurable outcomes expected (to confirm on the simulator/logs)

- Cross-track error during RTH/waypoint legs under steady crosswind → ~0 instead of the persistent downwind sag; fewer S-curves on the home leg.
- Orbits hold commanded radius instead of being pushed downwind.

Files

- UAVXArmQ/src/nav.c — WindCrabHeading(), constants NAV_WIND_CONF_MIN/NAV_WIND_MIN_GS_MPS/NAV_MAX_WIND_ wired in Navigate (FW branch) and DoOrbit. Forward-declared for DoOrbit.
- No params touched; FC recompile is the user's job. Build checked (BUILD OK, boot image FLYINGRCF4WINGMINIQ_r0.bin).

Follow-up (C, not done — touches the heading estimator)

Satellite **ground-track (course) coupler** into the heading estimate during autonomous FW cruise, to neutralise ESC/motor-current magnetometer error. Medium risk (inertial.c); only after A+B are logged flying. Also open from Section 6: wiring Wind into the static soar.c drift block.

Attribution: crab compensation technique adapted from the iNav project (flight/navigation.c wind compensation); attribution header in nav.c. Adapted 2026-08-20 by GKE.